

Automated Memory Forensics Pipeline — Detection Engineering Assessment

Volatility 3 Automation — Rootkit / Code-Injection Triage —
windows.pslist & windows.malfind — SIEM-Ready JSON Evidence

PREPARED BY	Ayush Kumar Patel
ENGAGEMENT TYPE	Self-Directed DFIR / SOC Home Lab Exercise
ENVIRONMENT	Kali Linux + Volatility 3 Framework 2.28.2
REPORT DATE	September 19, 2026
CLASSIFICATION	Internal / Portfolio Use

Table of Contents

- 1. Executive Summary
- 2. Engagement Scope & Objectives
- 3. Environment & Tooling
- 4. Methodology
- 5. Technical Walkthrough & Evidence
 - 5.1 Volatility 3 Engine Setup & Verification
 - 5.2 Automated Plugin Execution & Real-Time Detection
 - 5.3 Structured JSON Evidence Generation
- 6. Findings Summary
- 7. Risk Assessment
- 8. Recommendations
- 9. Conclusion

1. Executive Summary

This report documents a self-directed Digital Forensics & Incident Response (DFIR) exercise demonstrating an automated memory forensics triage pipeline built on top of the **Volatility 3 Framework**. The pipeline was executed against **cridex.vmem**, a well-known, publicly available memory sample containing the Cridex banking trojan, commonly used to validate memory-forensics tooling.

The pipeline automated two core plugins — **windows.pslist** (process enumeration) and **windows.malfind** (code-injection detection) — correlating their output in real time. The run successfully enumerated **17 running processes** and flagged **14 suspicious memory segments** exhibiting classic code-injection characteristics (pages marked **PAGE_EXECUTE_READWRITE**), triggering an immediate terminal alert and producing a timestamped, structured JSON report suitable for SIEM ingestion.

Sample Analyzed	cridex.vmem (public Cridex banking-trojan memory sample)
Processes Enumerated	17 (windows.pslist)
Suspicious Segments Flagged	14 (windows.malfind)
Example Injected Process	PID 584 — PAGE_EXECUTE_READWRITE
Detection Coverage	Achieved — real-time terminal alert + structured evidence
Evidence Output	./reports/threat_report_20260919_105627.json

2. Engagement Scope & Objectives

This was a self-contained lab exercise conducted against a publicly distributed malware memory sample (crindex.vmem), commonly used across the DFIR community to test and demonstrate memory-analysis tooling. No production systems, real victim data, or unauthorized targets were involved at any point.

Objectives

- Automate execution of core Volatility 3 triage plugins against a raw memory image.
- Programmatically correlate process-listing data with code-injection indicators.
- Trigger real-time, human-readable alerts the moment an anomaly is detected.
- Produce structured, timestamped JSON evidence suitable for downstream SIEM/TIP ingestion.
- Demonstrate a repeatable pipeline that scales beyond a single manual analyst workflow.

3. Environment & Tooling

Component	Detail
Operating System	Kali Linux
Forensics Framework	Volatility 3, Framework version 2.28.2
Automation Layer	Custom Python pipeline (src/advanced_pipeline.py)
Key Plugins Used	windows.pslist.PsList, windows.malware.malfind.Malfind
Sample Analyzed	cridex.vmem (public Cridex banking-trojan memory sample)
Evidence Format	Timestamped JSON (SIEM-ready)

4. Methodology

The engagement followed a standard automated-triage lifecycle, moving from raw memory acquisition to validated, structured findings:

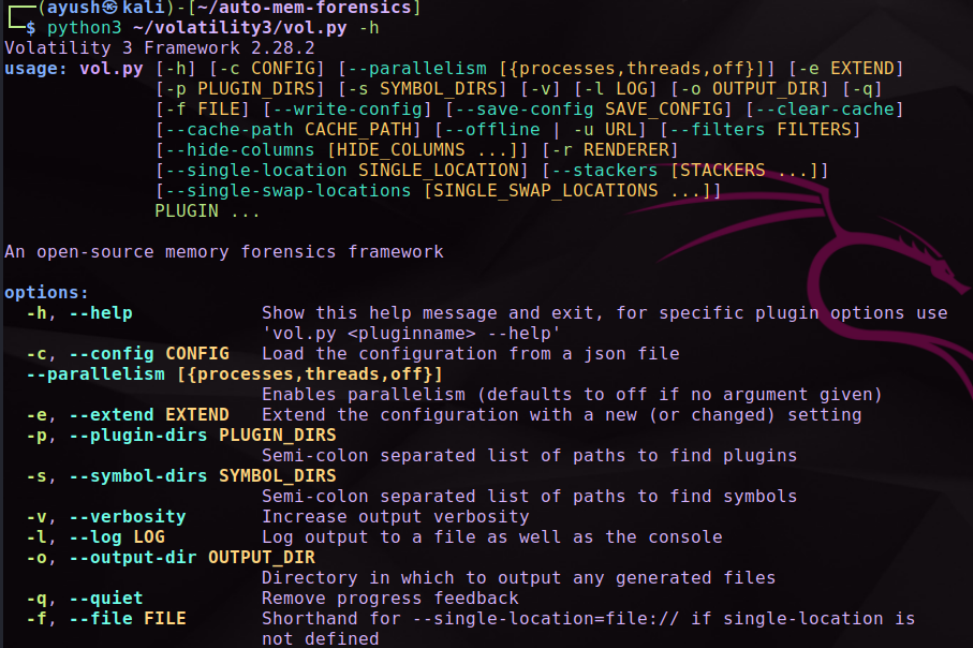
- **Verify** — confirm the Volatility 3 engine is correctly installed and responsive.
- **Enumerate** — run `windows.pslist` to build a full process tree from the memory image.
- **Inspect** — run `windows.malfind` to identify memory regions exhibiting code-injection characteristics.
- **Correlate** — cross-reference malfind hits back to their owning PID from the `pslist` output.
- **Alert** — raise an immediate terminal warning the moment a suspicious segment is found.
- **Preserve** — serialize all findings into a timestamped JSON report for downstream use.

5. Technical Walkthrough & Evidence

5.1 Volatility 3 Engine Setup & Verification

Before pointing the pipeline at a real memory dump, the Volatility 3 engine (Framework 2.28.2) was verified as correctly installed and responsive.

```
python3 ~/volatility3/vol.py -h
```



```
(ayush@kali) - [~/auto-mem-forensics]
$ python3 ~/volatility3/vol.py -h
Volatility 3 Framework 2.28.2
usage: vol.py [-h] [-c CONFIG] [--parallelism [{processes,threads,off}]] [-e EXTEND]
             [-p PLUGIN_DIRS] [-s SYMBOL_DIRS] [-v] [-l LOG] [-o OUTPUT_DIR] [-q]
             [-f FILE] [--write-config] [--save-config SAVE_CONFIG] [--clear-cache]
             [--cache-path CACHE_PATH] [--offline] [-u URL] [--filters FILTERS]
             [--hide-columns [HIDE_COLUMNS ...]] [-r RENDERER]
             [--single-location SINGLE_LOCATION] [--stackers [STACKERS ...]]
             [--single-swap-locations [SINGLE_SWAP_LOCATIONS ...]]
             PLUGIN ...

An open-source memory forensics framework

options:
  -h, --help                Show this help message and exit, for specific plugin options use
                           'vol.py <pluginname> --help'
  -c, --config CONFIG        Load the configuration from a json file
  --parallelism [{processes,threads,off}]
                           Enables parallelism (defaults to off if no argument given)
  -e, --extend EXTEND        Extend the configuration with a new (or changed) setting
  -p, --plugin-dirs PLUGIN_DIRS
                           Semi-colon separated list of paths to find plugins
  -s, --symbol-dirs SYMBOL_DIRS
                           Semi-colon separated list of paths to find symbols
  -v, --verbosity            Increase output verbosity
  -l, --log LOG              Log output to a file as well as the console
  -o, --output-dir OUTPUT_DIR
                           Directory in which to output any generated files
  -q, --quiet                Remove progress feedback
  -f, --file FILE            Shorthand for --single-location=file:// if single-location is
                           not defined
```

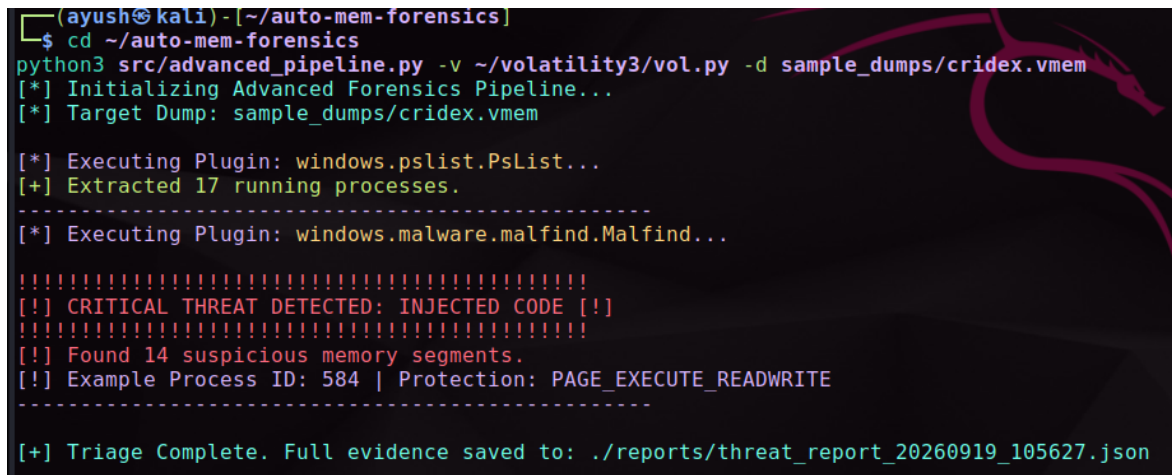
Figure 1 — Volatility 3 Framework 2.28.2 responding correctly to the help command, confirming the engine is ready before automation begins.

5.2 Automated Plugin Execution & Real-Time Detection

The pipeline was executed against the sample memory image, automatically chaining windows.pslist and windows.malfind in a single run.

```
python3 src/advanced_pipeline.py -v ~/volatility3/vol.py -d sample_dumps/cridex.vmem
```

windows.pslist extracted **17 running processes**. windows.malfind then flagged **14 suspicious memory segments** — regions marked **PAGE_EXECUTE_READWRITE**, a strong indicator of injected or unpacked malicious code, since legitimate processes rarely require memory that is simultaneously writable and executable. The pipeline immediately surfaced this via a real-time critical alert, citing PID **584** as an example offending process.



```
(ayush@kali) - [~/auto-mem-forensics]
$ cd ~/auto-mem-forensics
python3 src/advanced_pipeline.py -v ~/volatility3/vol.py -d sample_dumps/cridex.vmem
[*] Initializing Advanced Forensics Pipeline...
[*] Target Dump: sample_dumps/cridex.vmem

[*] Executing Plugin: windows.pslist.PsList...
[+] Extracted 17 running processes.
-----
[*] Executing Plugin: windows.malware.malfind.Malfind...

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[!] CRITICAL THREAT DETECTED: INJECTED CODE [!]
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[!] Found 14 suspicious memory segments.
[!] Example Process ID: 584 | Protection: PAGE_EXECUTE_READWRITE
-----

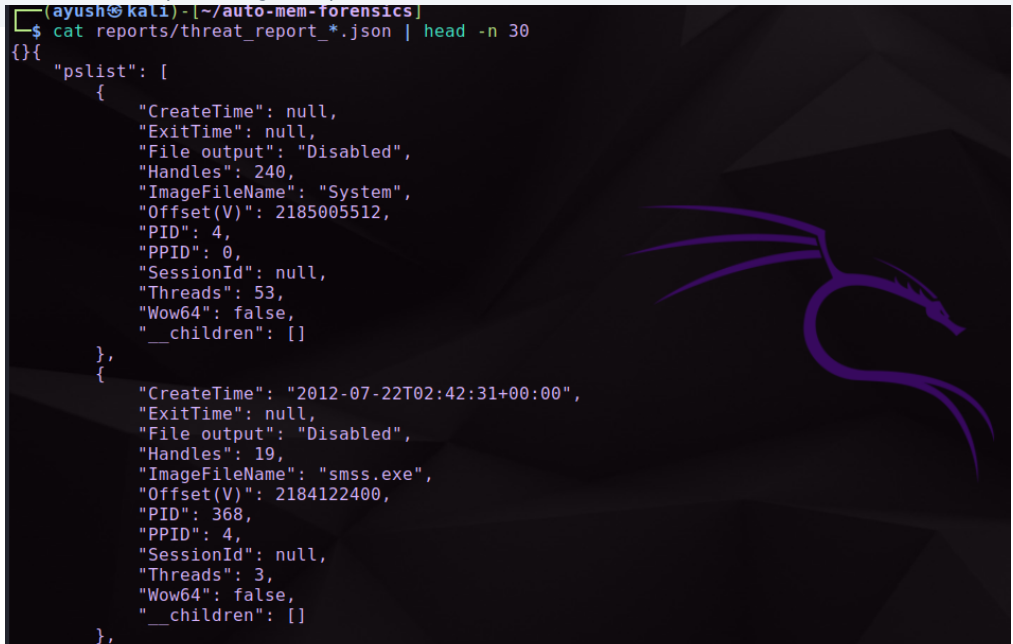
[+] Triage Complete. Full evidence saved to: ./reports/threat_report_20260919_105627.json
```

Figure 2 — Real-time pipeline output: 17 processes extracted, followed by a CRITICAL THREAT DETECTED alert after malfind identified 14 suspicious segments (example: PID 584, PAGE_EXECUTE_READWRITE).

5.3 Structured JSON Evidence Generation

On completion, the pipeline serialized the full pslist process tree and malfind findings into a timestamped JSON report, preserving every field (PID, PPID, handles, threads, create time) needed for further correlation or SIEM ingestion.

```
cat reports/threat_report_*.json | head -n 30
```



```
{
  "pslist": [
    {
      "CreateTime": null,
      "ExitTime": null,
      "File output": "Disabled",
      "Handles": 240,
      "ImageFileName": "System",
      "Offset(V)": 2185005512,
      "PID": 4,
      "PPID": 0,
      "SessionId": null,
      "Threads": 53,
      "Wow64": false,
      "__children": []
    },
    {
      "CreateTime": "2012-07-22T02:42:31+00:00",
      "ExitTime": null,
      "File output": "Disabled",
      "Handles": 19,
      "ImageFileName": "smss.exe",
      "Offset(V)": 2184122400,
      "PID": 368,
      "PPID": 4,
      "SessionId": null,
      "Threads": 3,
      "Wow64": false,
      "__children": []
    }
  ]
}
```

Figure 3 — Structured JSON evidence output showing the pslist process tree (e.g. System PID 4, smss.exe PID 368 with full metadata) — machine-readable and ready for downstream ingestion.

6. Findings Summary

#	Observation	Status
1	Volatility 3 engine verified operational prior to analysis.	Confirmed
2	windows.pslist successfully enumerated 17 processes from the memory image.	Confirmed
3	windows.malfind identified 14 memory segments with injection characteristics.	Confirmed
4	PID 584 flagged with PAGE_EXECUTE_READWRITE — consistent with code injection.	Confirmed
5	Real-time terminal alert triggered automatically without manual review.	Confirmed
6	Full findings preserved in a timestamped, SIEM-ready JSON report.	Confirmed

7. Risk Assessment

The Cridex banking trojan represented by this sample is a real, historically active family of malware known for credential theft and process injection. The 14 flagged memory segments — particularly the PAGE_EXECUTE_READWRITE region on PID 584 — are consistent with active code injection into a running process, a technique used to evade static, file-based antivirus detection entirely.

HIGH

Reflects the presence of confirmed code-injection indicators (writable+executable memory with no backing file) rather than a single ambiguous signal — this pattern alone typically warrants immediate escalation in a live environment.

Contributing Factors

- In-memory code injection leaves no reliable trace on disk, making memory acquisition the only dependable detection point.
- 14 flagged segments (out of 17 total processes) is a high ratio, suggesting broad injection activity rather than an isolated anomaly.
- Without an automated pipeline, manually correlating pslist and malfind output across dozens of processes is slow and error-prone during live incident response.

8. Recommendations

- **Isolate and further analyze PID 584** (and any other flagged PIDs) with additional plugins such as windows.dllexport and windows.handles.
- **Extend plugin coverage** to windows.netscan to check for injected processes establishing outbound connections.
- **Integrate the JSON output directly into a SIEM** via a log shipper, rather than reviewing reports manually.
- **Add a severity-scoring layer** so multiple simultaneous injection indicators (as seen here) automatically escalate priority.
- **Establish a memory-acquisition procedure** (e.g. via FTK Imager or avml) for all suspected-compromise hosts, since this entire class of threat is invisible to disk-only forensics.
- **Re-run this pipeline periodically** against new Volatility 3 releases to ensure plugin compatibility is maintained.

9. Conclusion

This exercise successfully demonstrated an automated memory-forensics triage pipeline capable of identifying real code-injection activity — enumerating 17 processes, flagging 14 suspicious memory segments, and producing structured, timestamped evidence without manual plugin chaining. Running the pipeline against a known malware sample (Cridex) validated that the detection logic correctly surfaces genuine injection indicators rather than benign noise.

The exercise reinforces a core DFIR principle: many of the most evasive threats leave their only reliable trace in volatile memory, and automating that analysis — rather than relying on slow, manual plugin chaining — is what makes memory forensics practical at incident-response speed.

Prepared by Ayush Kumar Patel as part of an ongoing cybersecurity / DFIR portfolio. This document is intended for educational and professional-portfolio purposes.